Arquivo: ./NearbyTestWatch Watch App/NearbyTestWatchApp.swift

```swift
//
//  NearbyTestWatchApp.swift
//  NearbyTestWatch Watch App
//
//  Created by João Cláudio dos Santos Souza on 05/11/26.
//

import SwiftUI

@main
struct NearbyTestWatch_Watch_AppApp: App {
    var body: some Scene {
        WindowGroup {
            ContentView()
        }
    }
}
```

Arquivo: ./NearbyTest/PeerConnection.swift

```swift
//
//  PeerConnection.swift
//  NearbyTest
//
//  Created by João Cláudio dos Santos Souza on 03/11/26.
//

import Foundation
import NearbyInteraction
import MultipeerConnectivity

class PeerConnection: NSObject, MCSessionDelegate,
MCNearbyServiceAdvertiserDelegate, MCNearbyServiceBrowserDelegate {

    let peerID: MCPeerID
    let communicationSession: MCSession
    var onPeerConnected: ((MCPeerID) -> Void)?
    var onPeerDisconnected: ((MCPeerID) -> Void)?
    var onTokenReceived: ((MCPeerID, NIDiscoveryToken) -> Void)?
    let advertiser: MCNearbyServiceAdvertiser
    let browser: MCNearbyServiceBrowser

    init(displayName: String) {
        let peer = MCPeerID(displayName: displayName)
        peerID = peer
        communicationSession = MCSession(peer: peer, securityIdentity: nil,
```

```
encryptionPreference: .required)
        advertiser = MCNearbyServiceAdvertiser(
            peer: peer,
            discoveryInfo: nil,
            serviceType: "quente-frio")
        browser = MCNearbyServiceBrowser(peer: peer, serviceType: "quente-
frio")
        super.init()
        communicationSession.delegate = self
        advertiser.delegate = self
        browser.delegate = self
    }

    func advertiser(_ advertiser: MCNearbyServiceAdvertiser,
didReceiveInvitationFromPeer peerID: MCPeerID, withContext context: Data?,
invitationHandler: @escaping (Bool, MCSession?) -> Void) {
        invitationHandler(true, communicationSession)
    }

    func browser(_ browser: MCNearbyServiceBrowser, foundPeer peerID:
MCPeerID, withDiscoveryInfo info: [String : String]?) {
        browser.invitePeer(
            peerID,
            to: self.communicationSession,
            withContext: nil,
            timeout: 30
        )
    }

    func browser(_ browser: MCNearbyServiceBrowser, lostPeer peerID:
MCPeerID) {

    }

    func startAdvertising() {
        advertiser.startAdvertisingPeer()
    }

    func startBrowsing() {
        browser.startBrowsingForPeers()
    }

    func session(_ session: MCSession, peer peerID: MCPeerID, didChange
state: MCSessionState) {
        DispatchQueue.main.async { [weak self] in
            guard let self else { return }

            print("Peer \(peerID.displayName): \(state)")
            print("Conectados:", session.connectedPeers.map(\.displayName))

            switch state {
            case .notConnected:
                self.onPeerDisconnected?(peerID)
            case .connecting:
```

```
                break
            case .connected:
                self.onPeerConnected?(peerID)
            @unknown default:
                break
            }
        }
    }

    func session(_ session: MCSession, didReceive data: Data, fromPeer
peerID: MCPeerID) {
        do {
            guard let token = try NSKeyedUnarchiver.unarchivedObject(
                ofClass: NIDiscoveryToken.self,
                from: data
            ) else {
                print("Dados recebidos não contém um token")
                return
            }

            print("Token recebido de \(peerID.displayName)")

            DispatchQueue.main.async { [weak self] in
                guard let self else { return }

                self.onTokenReceived?(peerID, token)
            }

            print("Token reconstruído: \(token)")
        } catch {
            print(error)
        }
    }

    func session(_ session: MCSession, didReceive stream: InputStream,
withName streamName: String, fromPeer peerID: MCPeerID) {

    }

    func session(_ session: MCSession, didStartReceivingResourceWithName
resourceName: String, fromPeer peerID: MCPeerID, with progress: Progress) {

    }

    func session(_ session: MCSession, didFinishReceivingResourceWithName
resourceName: String, fromPeer peerID: MCPeerID, at localURL: URL?,
withError error: (any Error)?) {

    }

    func trySendToken(token: NIDiscoveryToken, to peerID: MCPeerID) {
        do {
            let data = try NSKeyedArchiver.archivedData(
                withRootObject: token,
```

```
                requiringSecureCoding: true
            )

            try communicationSession.send(data, toPeers: [peerID], with:
.reliable)
        } catch {
            print(error.localizedDescription)
        }
    }
}
```

Arquivo: ./NearbyTest/NearbyManager.swift

```
//
//  NearbyManager.swift
//  NearbyTest
//
//  Created by João Cláudio dos Santos Souza on 02/11/26.
//

import Foundation
import NearbyInteraction
import UIKit
import MultipeerConnectivity

enum ProximityState {
    case waitingForMeasurement
    case nearby
    case farAway
}

enum DeviceSupportState {
    case waitingForVerification
    case supported
    case notSupported
}

@Observable
class NearbyManager: NSObject, NISessionDelegate {
    var sessionsByPeer: [MCPeerID: NISession] = [:]
    var peerBySession: [ObjectIdentifier: MCPeerID] = [:]
    var pendingTokensByPeer: [MCPeerID: NIDiscoveryToken] = [:]
    var proximityByPeer: [MCPeerID: ProximityState] = [:]

    var currentState: ProximityState = .waitingForMeasurement
    var currentDeviceSupportState: DeviceSupportState =
.waitingForVerification
    let connection = PeerConnection(displayName: "Jogador")

    override init() {
```

```
        super.init()

        connection.onPeerConnected = { [weak self] peerID in
            guard let self else { return }
            self.prepareSession(for: peerID)
        }

        connection.onTokenReceived = { [weak self] peerID, token in
            guard let self else { return }
            self.didReceivePeerToken(token, from: peerID)
        }

        connection.onPeerDisconnected = { [weak self] peerID in
            self?.removeSession(for: peerID)
        }
    }

    func prepareSession() {
        guard
NISession.deviceCapabilities.supportsPreciseDistanceMeasurement else {
            print("Dispositivo não suportado")
            currentDeviceSupportState = .notSupported
            return
        }

        currentDeviceSupportState = .supported
    }

    func prepareSession(for peerID: MCPeerID) {
        guard sessionsByPeer[peerID] == nil else { return }

        let session = NISession()
        session.delegate = self

        sessionsByPeer[peerID] = session
        peerBySession[ObjectIdentifier(session)] = peerID
        proximityByPeer[peerID] = .waitingForMeasurement

        guard let token = session.discoveryToken else { return }
        connection.trySendToken(token: token, to: peerID)

        if let pendingToken = pendingTokensByPeer[peerID] {
            pendingTokensByPeer[peerID] = nil
            didReceivePeerToken(pendingToken, from: peerID)
        }
    }

    func didReceivePeerToken(_ token: NIDiscoveryToken, from peerID:
MCPeerID) {
        guard let session = sessionsByPeer[peerID] else {
            pendingTokensByPeer[peerID] = token
            return
        }
```

```
        let config = NINearbyPeerConfiguration(peerToken: token)
        session.run(config)
    }

    func session(_ session: NISession, didUpdate nearbyObjects:
[NINearbyObject]) {
        guard let object = nearbyObjects.first else { return }
        guard let peerID = peerBySession[ObjectIdentifier(session)] else {
return }

        if let distance = object.distance {
            print("Distância: \(distance) m")
        } else {
            print("Distância Indisponível")
        }

        if let direction = object.direction {
            print("Direção: \(direction)")
        } else {
            print("Direção Indisponível")
        }

        let previousState: ProximityState

        if let savedState = proximityByPeer[peerID] {
            previousState = savedState
        } else {
            previousState = .waitingForMeasurement
        }

        let nextState = proximityState(for: object.distance, currentState:
previousState)
        proximityByPeer[peerID] = nextState
    }

    func removeSession(for peerID: MCPeerID) {
        pendingTokensByPeer[peerID] = nil
        proximityByPeer[peerID] = nil

        guard let session = sessionsByPeer[peerID] else { return }

        peerBySession[ObjectIdentifier(session)] = nil
        sessionsByPeer[peerID] = nil
        session.invalidate()
    }

    func proximityState(for distance: Float?, currentState: ProximityState)
-> ProximityState {
        guard let distance else { return .waitingForMeasurement }

        switch currentState {
        case .waitingForMeasurement:
            if distance < 1 {
                return .nearby
```

```
            } else {
                return .farAway
            }
        case .nearby:
            if distance >= 1.2 {
                return .farAway
            } else {
                return .nearby
            }
        case .farAway:
            if distance < 0.8 {
                return .nearby
            } else {
                return .farAway
            }
        }
    }
}
```

Arquivo: ./NearbyTest/ContentView.swift

```
//
//  ContentView.swift
//  NearbyTest
//
//  Created by João Cláudio dos Santos Souza on 02/11/26.
//

import SwiftUI
import MultipeerConnectivity

struct ContentView: View {

    @State private var manager = NearbyManager()

    var body: some View {
        VStack(spacing: 20) {
            Image(systemName: "globe")
                .imageScale(.large)
                .foregroundStyle(.tint)

            switch manager.currentDeviceSupportState {
            case .waitingForVerification:
                Text("Aguardando verificação")
            case .notSupported:
                Text("Dispositivo não suportado")
            case .supported:
                if manager.proximityByPeer.isEmpty {
                    Text("Aguardando jogadores")
                } else {
```

```
                    ForEach(Array(manager.proximityByPeer.keys), id:
\.self) { peerID in
                        if let state = manager.proximityByPeer[peerID] {
                            switch state {
                            case .farAway: Text("\(peerID.displayName):
Frio")
                            case .nearby: Text("\(peerID.displayName):
Quente")
                            case .waitingForMeasurement: Text("\
(peerID.displayName): Aguardando medição")
                            }
                        }
                    }
                }
            }

            Button("anunciar") {
                manager.connection.startAdvertising()
            }

            Button("Procurar Jogador") {
                manager.connection.startBrowsing()
            }
        }
        .padding()
        .onAppear {
            manager.prepareSession()
        }
    }
}

#Preview {
    ContentView()
}
```

Arquivo: ./NearbyTest/NearbyTestApp.swift

```
//
//  NearbyTestApp.swift
//  NearbyTest
//
//  Created by João Cláudio dos Santos Souza on 02/11/26.
//

import SwiftUI

@main
struct NearbyTestApp: App {
    var body: some Scene {
        WindowGroup {
```

```
                ContentView()
            }
        }
    }
```
